Pharmacy Management System (PMS_CS)

Project Documentation Report

Technology Stack:

C# WinForms | .NET Framework | ADO.NET | SQL Server

Architecture:

N-Tier (Models → Repositories → Services → Views)

Date: May 8th, 2026

Provided to:

Dr. Neveen Atef

Name	ID
Mohamed Khaled Ahmed	2201057
Mosa Abdulaziz Morgan	2200257
Youssef Yacoub Radi	2200649
Omar Mohamed Eid	2200284
Mohamed Ashraf Mohamed	2200597

Drive Link (Source code + SQL Queries + Report):
https://drive.google.com/drive/folders/1F_Jhhu5Pp92f-41R8nLqzrFp8QKbP2hd?usp=drive_link

Contents

1. Introduction & Project Theme 4
2. Database Architecture 4
2.1 Enhanced Entity-Relationship Diagram (EERD) 4
2.2 Relational Database Schema 5
2.3 Table Descriptions & Constraints 6
2.4 Initial Data Seeding 7
3. Application Interface (UI) 7
3.1 System Architecture Overview 7
3.2 Form Descriptions & Functionalities 7
EntryView 7
PatientLoginView 8
PatientSignupView 9
PatientProfileView 10
OrderView 11
PaymentView 12
PharmacistLoginView 13
PharmacistSignupView 13
PharmacistProfileView 14
InventoryView 14
ReceivedOrdersView 15
3.3 Navigation Flow & Diagram 16
4. Database Interactions (CRUD) 17
4.1 SELECT Operations 17
4.2 INSERT Operations 18
4.3 UPDATE Operations 18
4.4 DELETE / Soft-Delete Operations 19
5. Key Design Decisions 19
Appendix A: Foreign Key Constraints Reference 21
Appendix B: Sample SQL Scripts Overview 21

1. Introduction & Project Theme

The Pharmacy Management System (PMS_CS) is a desktop application designed to streamline the operations of a modern pharmacy. Developed using C# Windows Forms on the .NET Framework with ADO.NET data access, the system connects to a Microsoft SQL Server backend to provide a robust, role-aware management platform. The project supports three primary actor types: Patients (customers), Pharmacists, Cashiers, and Administrators.

The core objectives of the system are to: (a) maintain an accurate digital inventory of medicines with safety-effect tracking; (b) allow patients to place orders and settle payments using an internal wallet; (c) enable pharmacy staff to manage stock, process orders, and issue prescriptions; and (d) enforce business rules such as allergy-safety checks, stock availability guards, and role-based access control.

2. Database Architecture

2.1 Enhanced Entity-Relationship Diagram (EERD)

The EERD below illustrates the conceptual design of the PharmacyDB schema. It captures the inheritance hierarchy of USER into PATIENT and EMPLOYEE (specialized further into ADMIN, CASHIER, and PHARMACIST), the multi-valued attributes (Allergies, Side_Effects, Healing_Effects), and the relationships between orders, prescriptions, medicines, and payments.

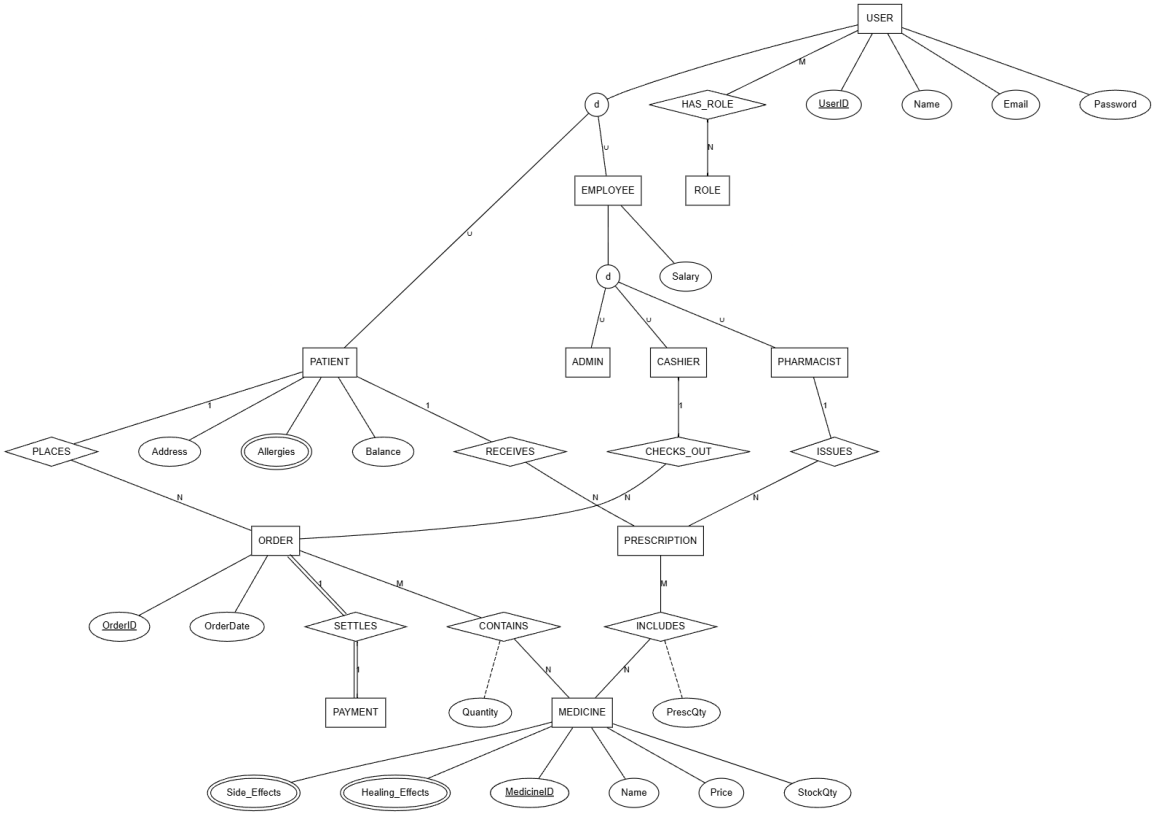


Figure 1: Enhanced Entity-Relationship Diagram (EERD) of PharmacyDB

Key conceptual observations from the EERD: (1) USER is the super-type; PATIENT and EMPLOYEE are sub-types linked by shared primary/foreign keys (UserID). (2) Multi-valued attributes are normalized into separate tables (PATIENT_ALLERGY, MED_SIDE_EFFECT, MED_HEALING_EFFECT). (3) ORDER and PRESCRIPTION are weak entities dependent on PATIENT and EMPLOYEE, respectively, with ORDER_ITEM and PRESCRIPTION_ITEM resolving M: N relationships between orders/prescriptions and medicines.

2.2 Relational Database Schema

The logical schema was derived directly from the EERD with full normalization. Primary keys are underlined.

```
USER(UserID, Username, Password, Email, Phone, IsActive)
ROLE(RoleID, RoleName, PermissionsLevel, Description)
USER_ROLE(UserID, RoleID)
PATIENT(UserID, Age, Address, PatientBalance)
PATIENT_ALLERGY(UserID, AllergyName)
EMPLOYEE(UserID, Salary, JobType)
MEDICINE(MedicineID, Name, Price, Category, ExpiryDate,
StockQuantity, UsageInstructions, IsRefundable)
MED_SIDE_EFFECT(MedicineID, SideEffectName)
```

```
MED_HEALING_EFFECT(MedicineID, HealingEffectName)
ORDER(OrderID, OrderDate, TotalPrice, Status, PatientID,
CashierID)
ORDER_ITEM(OrderID, MedicineID, Quantity, UnitPrice)
PRESCRIPTION(PrescriptionID, IssueDate, Status, PatientID,
PharmacistID)
PRESCRIPTION_ITEM(PrescriptionID, MedicineID, PrescribedQuantity)
PAYMENT(PaymentID, Amount, PaymentDate, PaymentMethod, Status,
OrderID)
```

2.3 Table Descriptions & Constraints

- **USER:** Central identity table. Stores login credentials and contact information. UserID is an IDENTITY primary key. IsActive defaults to 1 (true), supporting soft-deactivation.
- **ROLE:** Lookup table for authorization roles. PermissionsLevel provides a numeric hierarchy (e.g., Admin=99, Pharmacist=50, Cashier=10).
- **USER_ROLE:** Associative entity resolving the M:N relationship between users and roles. Composite PK (UserID, RoleID).
- **PATIENT:** Subtype of USER. UserID serves as both PK and FK to USER. Stores demographic and financial balance data. PatientBalance defaults to 0.0.
- **PATIENT_ALLERGY:** Multi-valued attribute table. Composite PK (UserID, AllergyName) ensures no duplicate allergies per patient.
- **EMPLOYEE:** Subtype of USER. UserID is PK/FK. JobType is constrained via CHECK to 'Admin', 'Cashier', or 'Pharmacist'. Salary defaults to 0.
- **MEDICINE:** Core inventory table. MedicineID is IDENTITY PK. StockQuantity defaults to 0. IsRefundable is a BIT flag.
- **MED_SIDE_EFFECT:** Multi-valued attribute table for adverse effects. Composite PK prevents duplicate side-effect entries per medicine.
- **MED_HEALING_EFFECT:** Multi-valued attribute table for therapeutic effects. Same composite PK pattern as side effects.
- **ORDER:** Transaction header. OrderID is IDENTITY PK. Status is CHECK constrained to 'Pending', 'Completed', or 'Cancelled'. TotalPrice defaults to 0.
- **ORDER_ITEM:** Line-item detail for orders. Composite PK (OrderID, MedicineID). UnitPrice is snapshotted at time of order to preserve historical pricing.
- **PRESCRIPTION:** Medical authorization header. PrescriptionID is IDENTITY PK. Status is CHECK constrained to 'Active', 'Filled', 'Expired', or 'Cancelled'.
- **PRESCRIPTION_ITEM:** Line-item detail for prescriptions. Composite PK (PrescriptionID, MedicineID). PrescribedQuantity stores the dosage amount.
- **PAYMENT:** Financial settlement record. PaymentID is IDENTITY PK. Status defaults to 'Pending' with CHECK constraint. OrderID FK links to exactly one order.

2.4 Initial Data Seeding

The database is seeded with representative data across all tables to demonstrate functionality. Three patients, three employees (Admin, Cashier, Pharmacist), three medicines, three orders, three prescriptions, and three payments are inserted. Additionally, multi-valued attributes (allergies, side effects, healing effects) and role assignments are populated to ensure every table contains at least three rows as per project requirements.

3. Application Interface (UI)

3.1 System Architecture Overview

The application follows a strict N-tier (layered) architecture to separate concerns and promote maintainability:

- **Presentation Layer (Views):** WinForms UserControls hosted in a MainForm shell. Each view is responsible for layout, event handling, and calling service methods. No direct SQL exists in this layer.
- **Business Logic Layer (Services):** Encapsulates domain rules: order validation, balance checks, stock deductions, allergy-safety verification, and role authorization. Services coordinate multiple repositories when a business process spans tables.
- **Data Access Layer (Repositories):** Direct ADO.NET interaction using SqlConnection, SqlCommand, and SqlDataReader. Each repository maps to one primary entity. Parameterized queries are used exclusively to prevent SQL injection.
- **Model Layer (Entities):** Plain C# classes (POCOs) representing database rows. They contain primitive properties, navigation properties (e.g., UserInfo), and lightweight business methods (e.g., CanAfford, IsExpired).

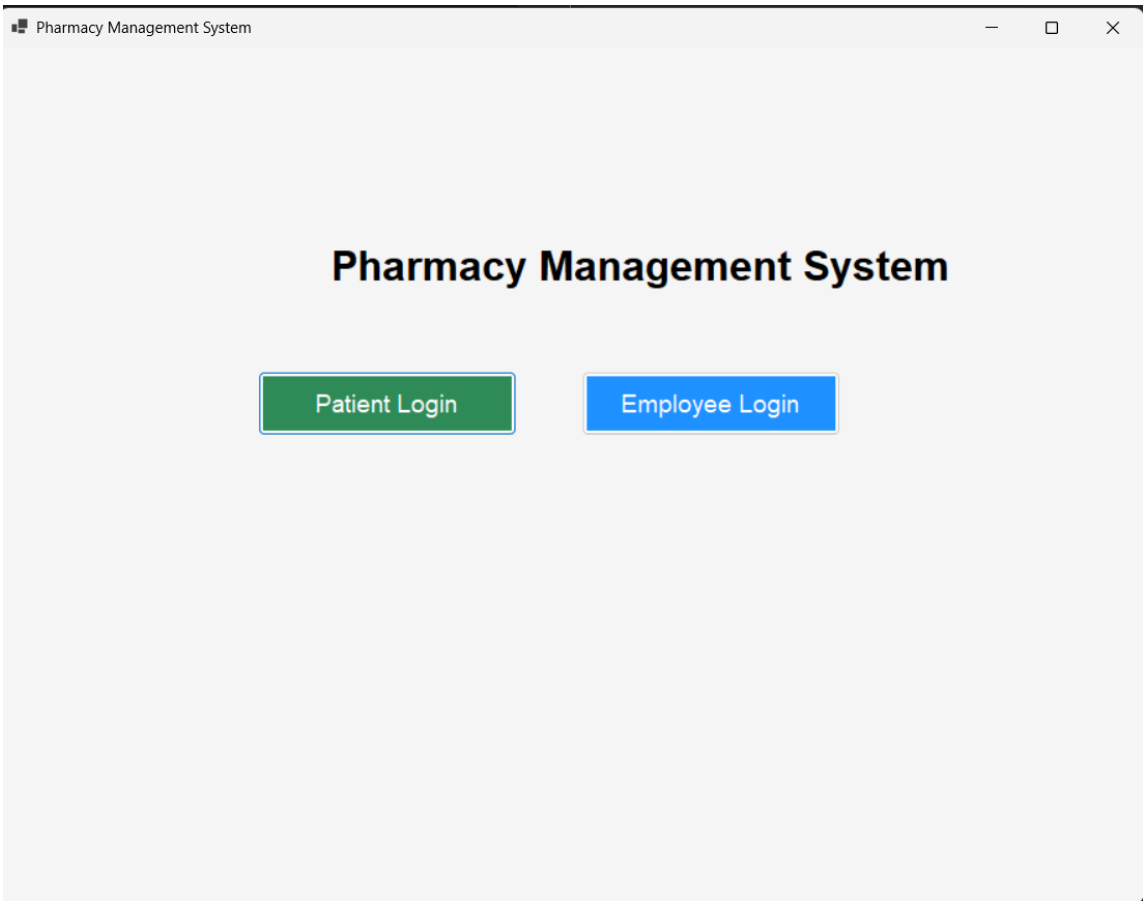
A static Session class maintains the authenticated user's state across views, storing CurrentUser, CurrentPatient, and CurrentEmployee objects. Navigation is orchestrated by MainForm.LoadPage(), which swaps UserControls in a docked Panel.

3.2 Form Descriptions & Functionalities

The application implements eleven distinct views, each serving a specific purpose in the workflow.

EntryView

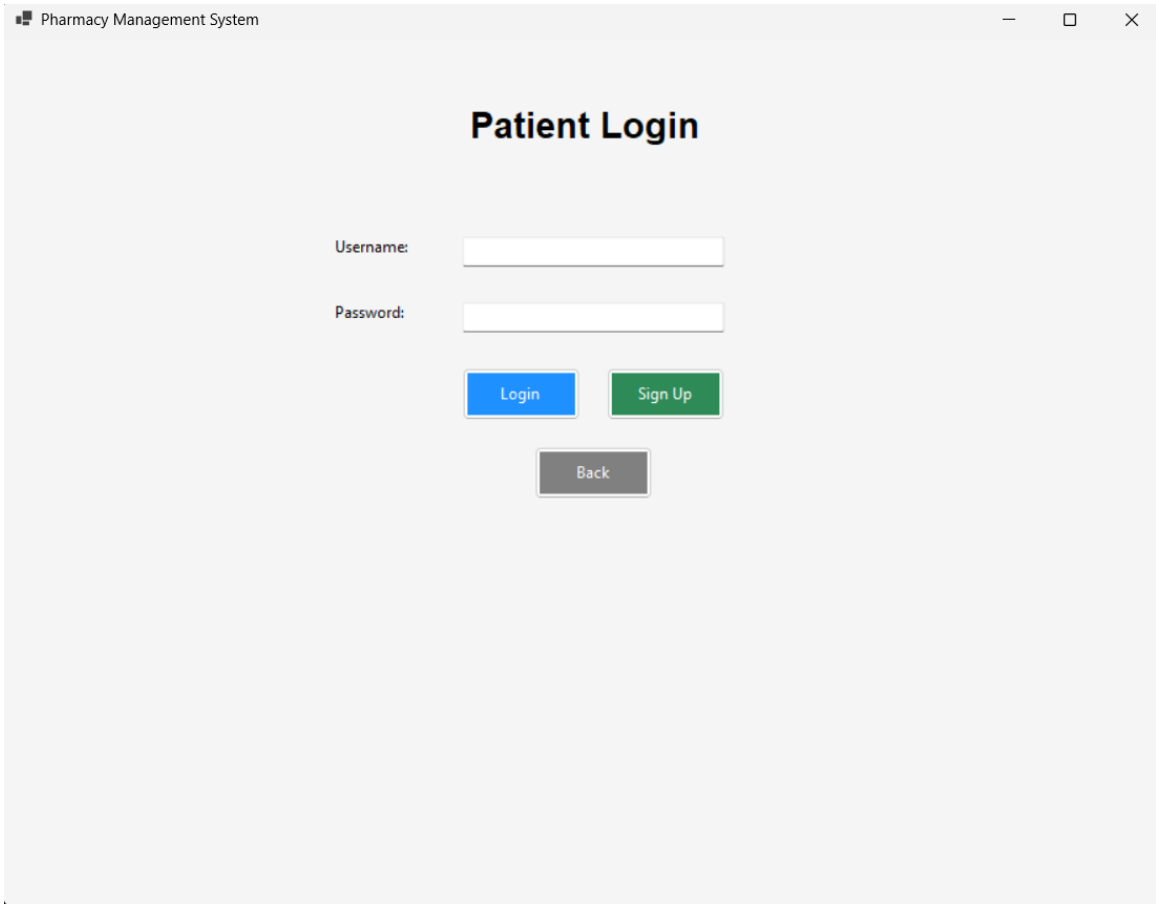
Purpose & Functionality: The landing screen presented on application startup. It displays the system title and provides two primary pathways: 'Patient Login' (navigates to PatientLoginView) and 'Employee Login' (navigates to PharmacistLoginView). This view acts as the root of the navigation graph.



Database Operations: Navigation hub; no database interaction.

PatientLoginView

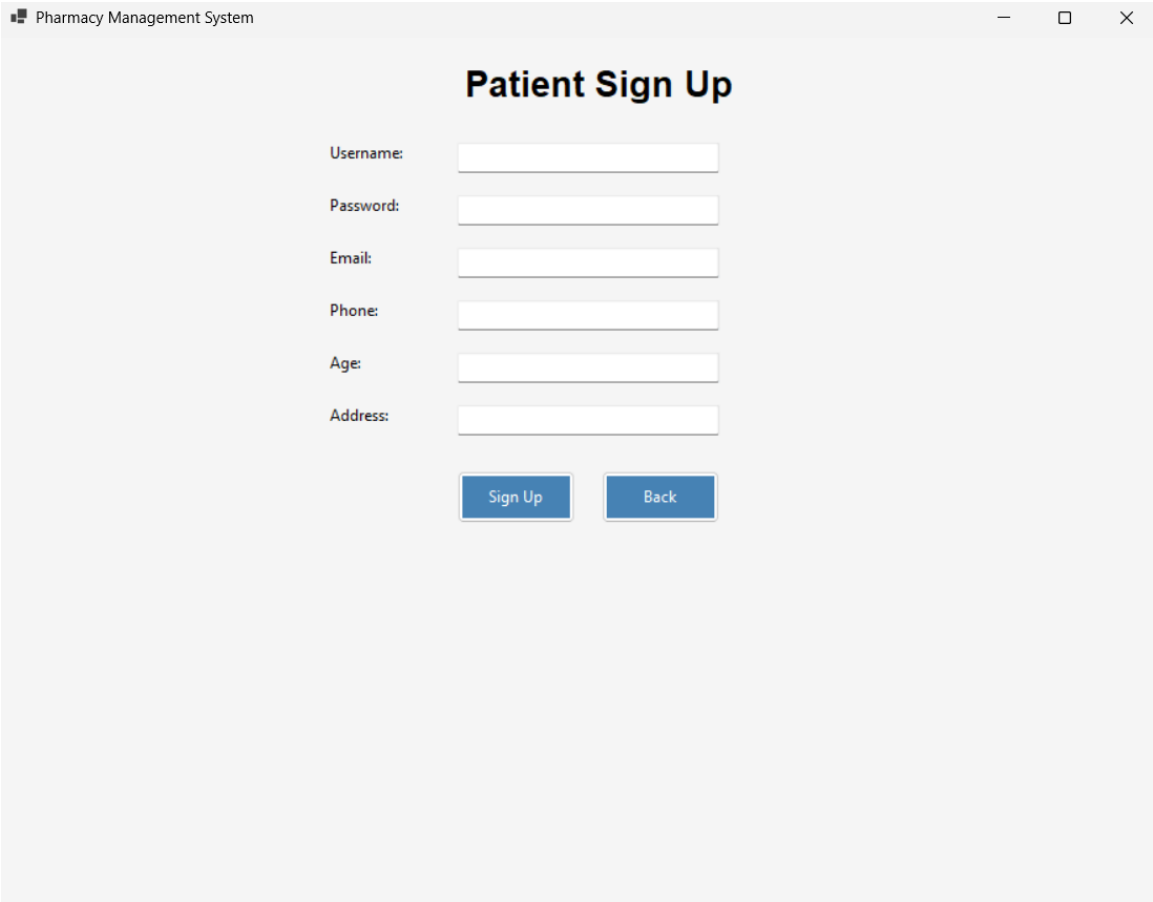
Purpose & Functionality: Collects Username and Password via TextBox controls. On submit, it invokes UserService.Login to validate credentials, then load the patient profile via PatientService.GetPatientById(). If authentication succeeds, the Session is populated, and the user is forwarded to PatientProfileView; otherwise, an error MessageBox is displayed. A 'Sign Up' button routes to PatientSignupView.



Database Operations: SELECT (USER, PATIENT).

PatientSignupView

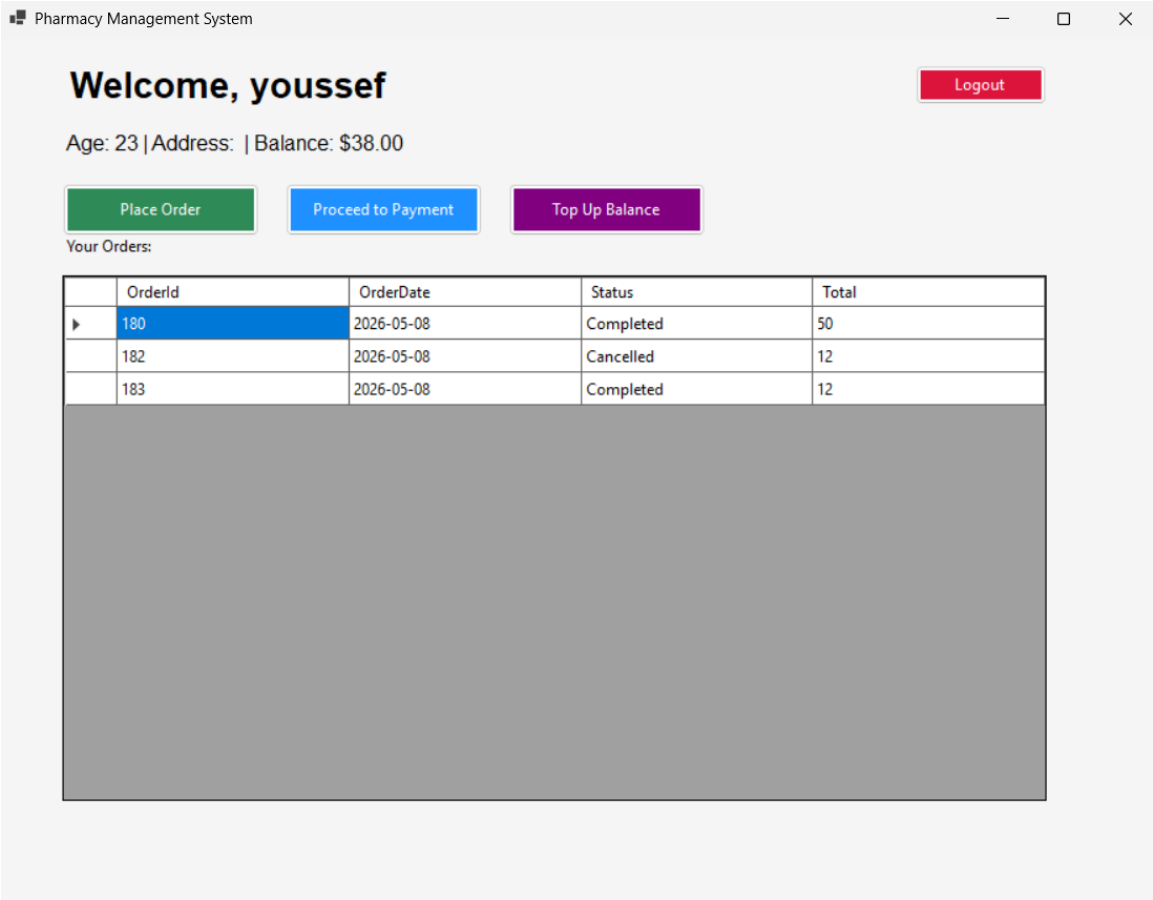
Purpose & Functionality: Registration form for new patients. Captures Username, Password, Email, Phone, Age, and Address. It delegates to UserService.RegisterPatient(), which performs a transactional two-step insert: first into USER (returning the new IDENTITY UserID), then into PATIENT. On success, the system attempts an automatic login and redirects to PatientProfileView.



Database Operations: INSERT (USER, PATIENT).

PatientProfileView

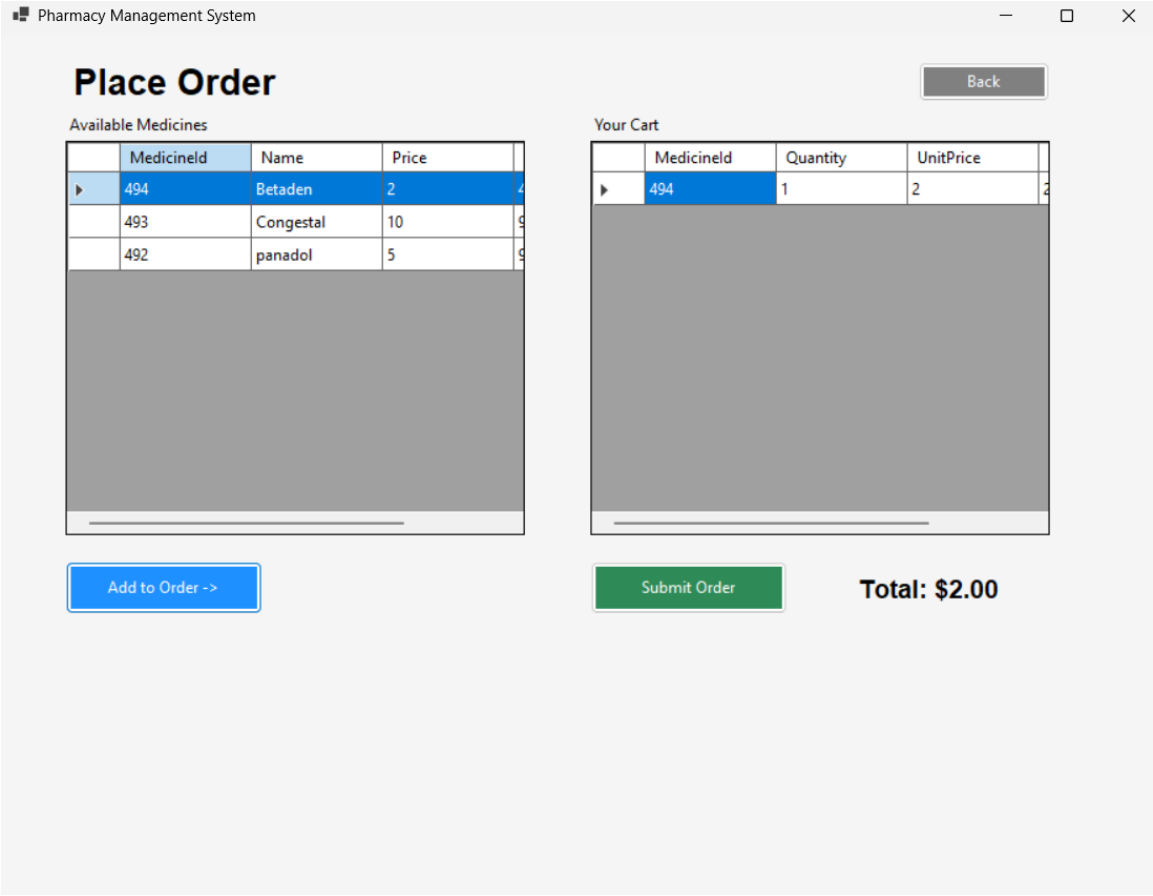
Purpose & Functionality: The patient dashboard. Displays a welcome label, demographic summary (Age, Address, Balance), and a DataGridView of the patient's order history populated via OrderService.GetOrdersByPatient(). Action buttons include: 'Place Order' → OrderView, 'Proceed to Payment' → PaymentView, 'Top Up Balance' (prompts for amount and calls PatientService.TopUpBalance()), and 'Logout' (clears Session).



Database Operations: SELECT (ORDER); UPDATE (PATIENT balance).

OrderView

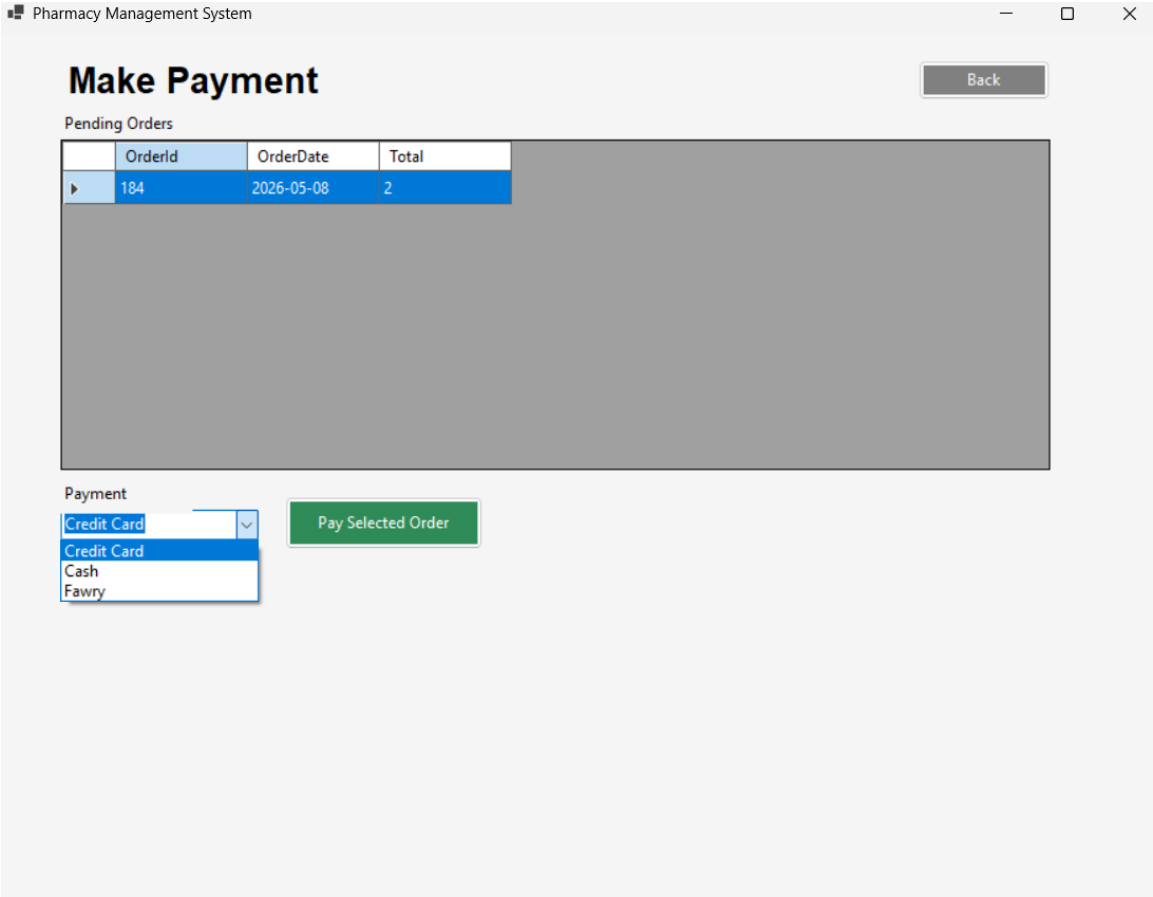
Purpose & Functionality: A split-pane ordering interface. The left DataGridView lists available medicines from InventoryService.GetAllMedicines(). The user selects a row, enters a quantity via a custom Prompt dialog, and clicks 'Add to Order' to build an in-memory cart. The right DataGridView displays the running cart (currentOrder.OrderItems). The total is recalculated dynamically. 'Submit Order' assigns the current patient and the first available Cashier/Admin as the CashierID, then calls OrderService.PlaceOrder().



Database Operations: SELECT (MEDICINE); INSERT (ORDER, ORDER_ITEM); UPDATE (MEDICINE stock, PATIENT balance).

PaymentView

Purpose & Functionality: Displays all pending orders for the logged-in patient in a DataGridView. The user selects an order and chooses a payment method (Credit Card, Cash, or Fawry) from a ComboBox. On 'Pay Selected Order', PaymentService.ProcessPayment() creates a PAYMENT record, marks it 'Completed', and updates the corresponding ORDER status to 'Completed'. The patient's balance is refreshed in Session.



Database Operations: SELECT (ORDER); INSERT (PAYMENT); UPDATE (ORDER status, PATIENT balance).

PharmacistLoginView

Purpose & Functionality: Employee authentication screen. Validates credentials via UserService.Login(), then loads the employee record via EmployeeRepository. It accepts any valid employee role (Admin, Cashier, or Pharmacist) and routes to PharmacistProfileView. Includes a 'Sign Up' button for new staff.

Database Operations: SELECT (USER, EMPLOYEE).

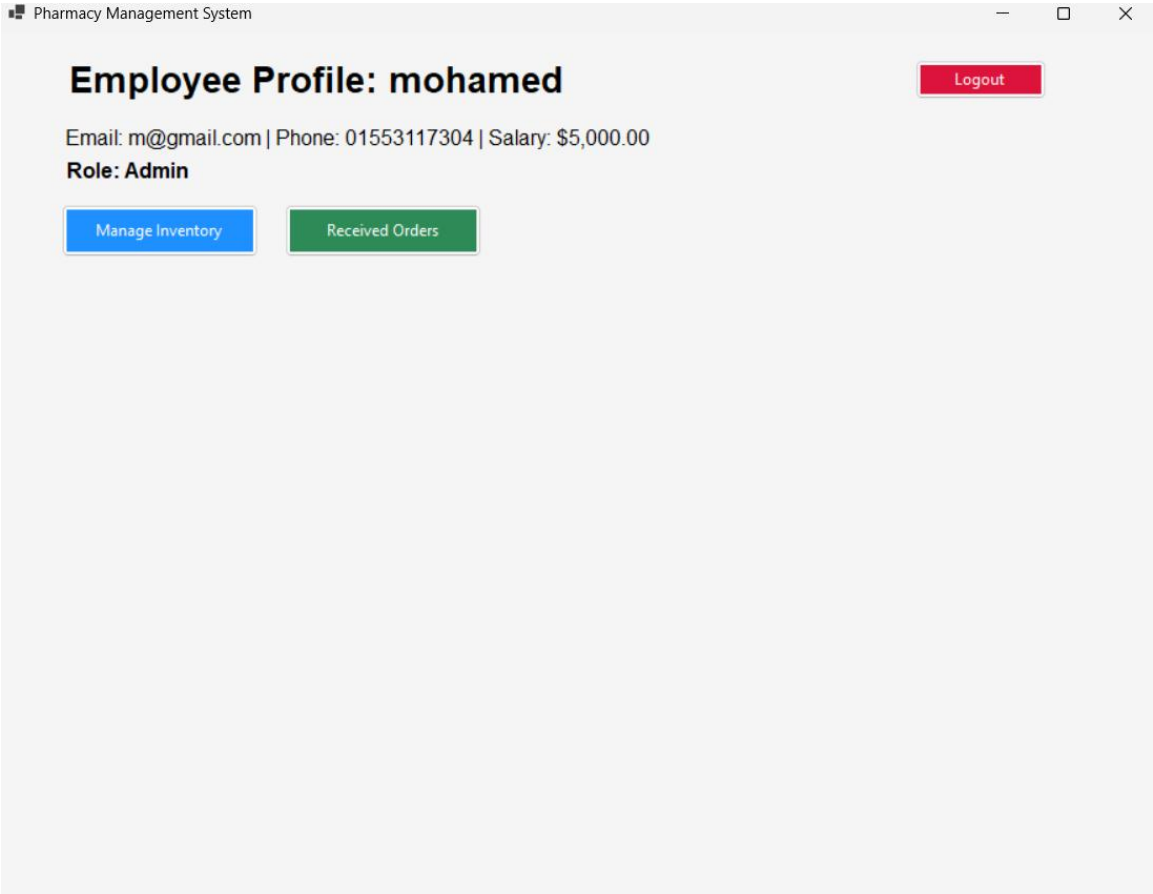
PharmacistSignupView

Purpose & Functionality: Employee registration gated by a validation code ('PHARMACY123'). Captures Username, Password, Email, Phone, and Role (Pharmacist or Cashier). Because the backend requires an Admin to register employees, the view temporarily simulates an Admin context to satisfy the service-layer authorization check. Calls UserService.RegisterEmployee() which inserts into USER and EMPLOYEE tables.

Database Operations: INSERT (USER, EMPLOYEE).

PharmacistProfileView

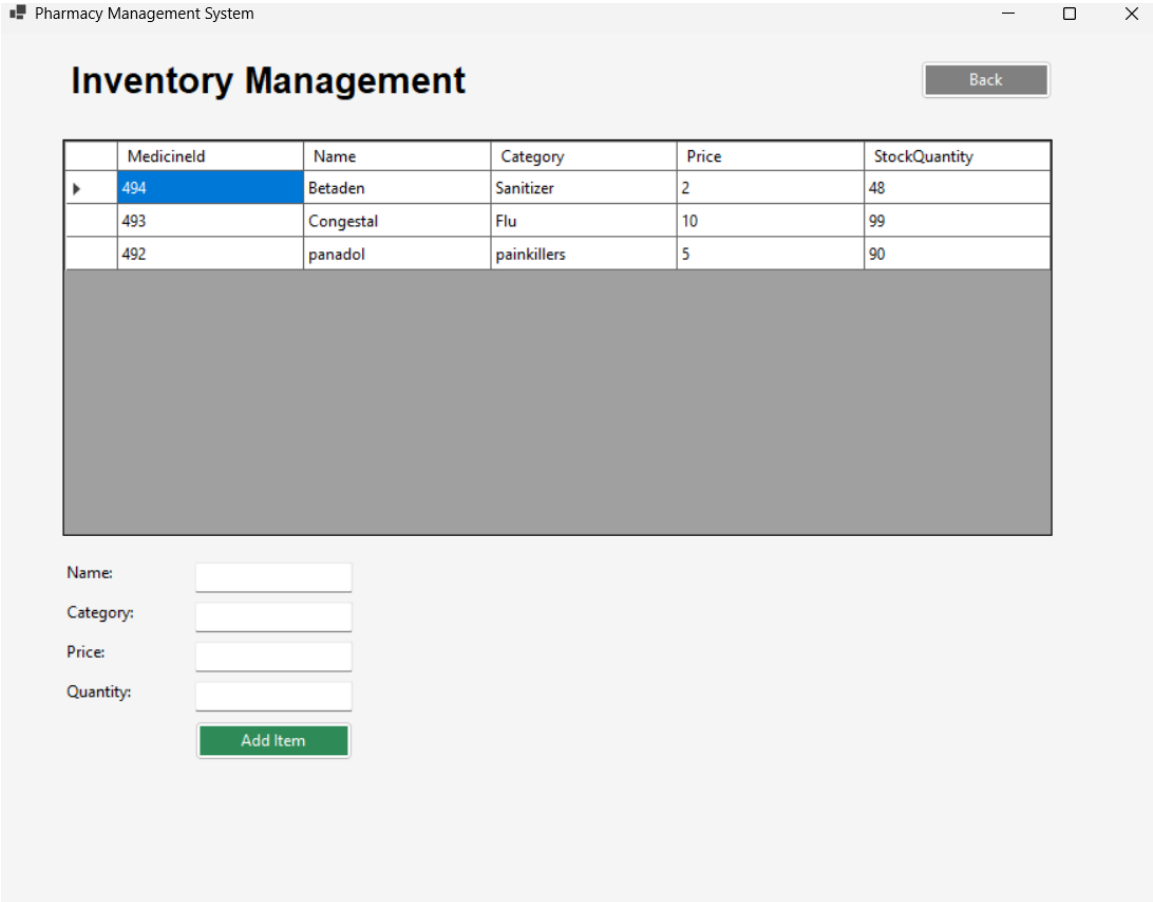
Purpose & Functionality: Role-aware employee dashboard. Displays username, email, phone, salary, and job type. Buttons are conditionally enabled: 'Manage Inventory' is active for Pharmacists and Admins; 'Received Orders' is active for Cashiers and Admins. This demonstrates granular, role-based UI rendering.



Database Operations: Navigation hub; SELECT (EMPLOYEE metadata).

InventoryView

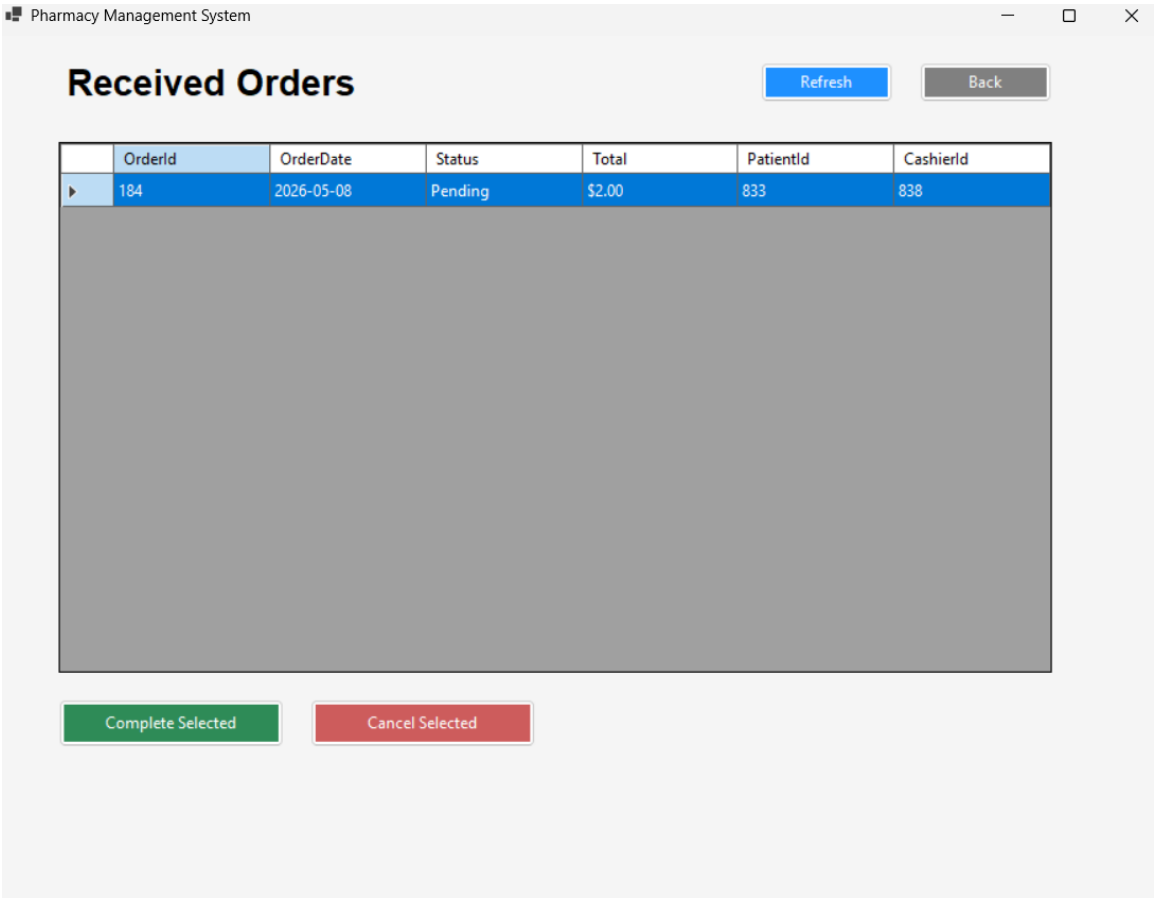
Purpose & Functionality: Stock management interface. A DataGridView shows all medicines (ID, Name, Category, Price, Stock). Below the grid, input fields allow the user to add a new medicine (Name, Category, Price, Quantity). The 'Add Item' button invokes InventoryService.AddMedicine(), which enforces that only Admins may insert new medicines. A 'Back' button returns to PharmacistProfileView.



Database Operations: SELECT (MEDICINE); INSERT (MEDICINE).

ReceivedOrdersView

Purpose & Functionality: Order fulfillment workspace for Cashiers and Admins. Displays pending orders in a DataGridView with Refresh, Complete Selected, and Cancel Selected actions. CompleteOrder() verifies the employee is a Cashier or Admin, then updates the order status. CancelOrder() performs a business-level cancellation: it updates status to 'Cancelled', restores medicine stock via InventoryService.RestoreStock(), and refunds the patient's balance.



Database Operations: SELECT (ORDER); UPDATE (ORDER status); UPDATE/DELETE logic (stock restore, balance refund).

3.3 Navigation Flow & Diagram

Navigation is centralized through MainForm.LoadPage(), ensuring a single-page application (SPA) experience within a desktop WinForms host. The diagram below describes the valid state transitions between views:

```
[EntryView]
    |--(Patient Login)--> [PatientLoginView] --(Success)--> [PatientProfileView]
    |                           ^                                |
    |                           |                                |--(Place Order)-->
[OrderView] --------|
    |                           |                                |--(Payment)----->
[PaymentView] ------|
    |                           |                                |--(Top Up)------>
(Dialog, returns)   |
    |                           |                                |--(Logout)------>
[EntryView]         |
    |--(Employee Login)-> [PharmacistLoginView] --(Success)--> [PharmacistProfileView]
    |                           ^                                |
    |                           |                                |--(Inventory)---->
[InventoryView] ---|
    |                           |                                |--(Orders)------->
[ReceivedOrdersView]
    |                           |                                |--(Logout)------->
[EntryView]        |
    |--(Patient Signup)-> [PatientSignupView] --(Success)----> [PatientProfileView]
    |--(Employee Signup)-> [PharmacistSignupView] --(Success)-> [PharmacistProfileView]
```

The navigation graph is acyclic with the exception of the back/logout edges returning to EntryView or the parent profile. Session state determines conditional rendering (e.g., button enablement in PharmacistProfileView) and access control gates at the service layer.

4. Database Interactions (CRUD)

The application satisfies all four required database operation categories. Below is a detailed breakdown of where and how each operation is exercised, including the specific ADO.NET patterns employed (parameterized commands, OUTPUT INSERTED for identity retrieval, and explicit SqlTransactions).

4.1 SELECT Operations

- **Authentication:** UserRepository.Authenticate() performs a SELECT on USER filtering by Username, Password, and IsActive=1. Returns a fully populated User model.
- **Patient Profile Loading:** PatientRepository.GetPatientById() executes a JOIN between PATIENT and USER to retrieve demographics and account status in a single round-trip.
- **Inventory Browsing:** MedicineRepository.GetAllMedicines() and GetMedicinesByCategory() return Medicine lists. Side effects and healing effects are loaded on-demand via separate queries.
- **Order History:** OrderRepository.GetOrdersByPatient() and GetOrdersByCashier() filter ORDER rows by foreign key and sort descending by date.
- **Pending Order Queue:** OrderRepository.GetPendingOrders() is used by ReceivedOrdersView to show only orders awaiting fulfillment.
- **Payment Verification:** PaymentRepository.GetPaymentByOrder() guards against double-payment by checking for an existing PAYMENT row before inserting a new one.

- **Allergy & Safety Checks:** PatientRepository.GetAllergies() and MedicineRepository.GetSideEffects() feed into InventoryService.IsMedicineSafeForPatient(), which uses LINQ Any() to detect conflicts.
- **Role Resolution:** UserRepository.GetRolesForUser() JOINs ROLE and USER_ROLE to populate the user's authorization list after login.

4.2 INSERT Operations

- **Patient Registration:** UserRepository.AddUser() uses OUTPUT INSERTED.UserID to capture the new identity, which is then passed to PatientRepository.AddPatient() to satisfy the PK/FK relationship.
- **Employee Registration:** UserRepository.AddUser() followed by EmployeeRepository.AddEmployee(). The service layer enforces that only Admins can invoke this workflow (with a UI bypass for standalone signup).
- **Order Placement:** OrderRepository.AddOrder() wraps the header INSERT and all ORDER_ITEM INSERTs inside a SqlTransaction. If any item fails, Rollback() prevents partial orders. The new OrderID is returned via OUTPUT INSERTED.
- **Payment Recording:** PaymentRepository.AddPayment() inserts a PAYMENT row with status 'Pending', then immediately updates it to 'Completed' upon success.
- **Prescription Issuance:** PrescriptionRepository.AddPrescription() similarly uses a transaction to write the PRESCRIPTION header and its PRESCRIPTION_ITEM lines atomically.
- **Inventory Expansion:** MedicineRepository.AddMedicine() inserts the MEDICINE row, captures the new MedicineID, and then iterates to insert associated side effects and healing effects into their respective multi-value tables.
- **Allergy Registration:** PatientRepository.AddAllergy() uses an IF NOT EXISTS guard to prevent duplicate allergy entries.

4.3 UPDATE Operations

- **Order Status Transition:** OrderRepository.UpdateStatus() changes ORDER.Status to 'Completed' or 'Cancelled'. Business rules in OrderService prevent illegal transitions (e.g., completing an already cancelled order).
- **Patient Balance Adjustment:** PatientRepository.UpdateBalance() is called during top-up (Credit), order placement (Debit), order cancellation (Refund Credit), and payment processing.
- **Stock Adjustment:** MedicineRepository.UpdateStock() accepts a signed integer delta. A database-level guard (StockQuantity + @Delta >= 0) prevents negative inventory.
- **Medicine Metadata Edit:** MedicineRepository.UpdateMedicine() modifies name, price, category, expiry, stock, usage instructions, and refundability. Restricted to Admin users by InventoryService.
- **Employee Salary Update:** EmployeeRepository.UpdateSalary() is exposed through UserService.SetEmployeeSalary() with an Admin authorization check.

- **User Profile & Password:** UserRepository.UpdateUser() edits contact info; ChangePassword() isolates credential updates. Both return boolean success based on ExecuteNonQuery() == 1.
- **Payment Status Finalization:** PaymentRepository.UpdateStatus() moves a payment from 'Pending' to 'Completed' after the financial record is successfully persisted.

4.4 DELETE / Soft-Delete Operations

The system employs a mix of hard deletion, business-level cancellation, and soft deactivation depending on referential integrity constraints and audit requirements.

- **Medicine Hard Delete:** MedicineRepository.DeleteMedicine() executes a DELETE FROM MEDICINE. Referential integrity constraints on ORDER_ITEM and PRESCRIPTION_ITEM will block deletion if the medicine is referenced by historical transactions, preventing accidental data loss.
- **Order Cancellation (Business Delete):** OrderService.CancelOrder() does not remove the ORDER row; instead it sets Status='Cancelled', restores stock quantities via InventoryService.RestoreStock(), and refunds the patient balance. This preserves the audit trail while reversing the business effect.
- **Prescription Cancellation:** PrescriptionService.CancelPrescription() updates status to 'Cancelled' but only if the prescription has not yet been filled, protecting dispensing records.
- **User Deactivation (Soft Delete):** UserRepository.DeactivateUser() sets IsActive=0 rather than deleting the USER row. This preserves all associated orders, prescriptions, and payments while preventing future logins.
- **Role Revocation:** UserRepository.RemoveRoleFromUser() deletes the specific USER_ROLE associative row, effectively removing a user's permission without deleting the user or role.

5. Key Design Decisions

- **Parameterized Queries & SQL Injection Prevention:** Every repository method uses SqlParameter to bind variables. No string concatenation is used in SQL command construction, eliminating SQL injection vectors.
- **Transaction Boundaries:** OrderRepository.AddOrder() and PrescriptionRepository.AddPrescription() use SqlTransaction with explicit Commit() and Rollback(). This ensures that a header row and its line items are written atomically; partial writes are impossible.
- **Role-Based Access Control (RBAC):** Authorization is enforced at two levels: (1) the UI layer disables buttons based on Employee.JobType (e.g., PharmacistProfileView), and (2) the service layer re-validates roles before executing sensitive operations (e.g., InventoryService.AddMedicine requires IsAdmin()). This defense-in-depth pattern prevents bypass attacks.

- **On-Demand vs. Eager Loading:** Multi-valued attributes (allergies, side effects) and navigation properties (UserInfo) are loaded on-demand by repositories rather than via ORM lazy loading. This gives the developer explicit control over query timing and avoids the N+1 query problem in list views.
- **Session State Management:** A static Session class holds CurrentUser, CurrentPatient, and CurrentEmployee. While static state is not ideal for multi-threaded or multi-user deployments, it is appropriate for a single-instance desktop application and simplifies view-to-view data passing without constructor proliferation.
- **Separation of Concerns:** Models contain data and trivial computed properties (e.g., LineTotal). Repositories contain SQL. Services contain business rules (e.g., CanAfford, IsExpired checks). Views contain layout and event wiring. This separation makes unit testing and maintenance significantly easier.
- **Database Name Configuration:** The connection string in DBConnection.cs targets 'PharmacyDB_Test' with Integrated Security and TrustServerCertificate=True for local development. The deployment SQL scripts reference 'PharmacyDB'. For production, this string should be externalized to an app.config or environment variable.
- **Validation Code for Employee Signup:** PharmacistSignupView requires the code 'PHARMACY123' to prevent unauthorized employee account creation. This is a lightweight alternative to email-based invitation workflows suitable for a classroom project scope.

Appendix A: Foreign Key Constraints Reference

The following foreign key constraints enforce referential integrity across the PharmacyDB schema:

1. USER_ROLE.UserID → USER.UserID
2. USER_ROLE.RoleID → ROLE.RoleID
3. PATIENT.UserID → USER.UserID (PK is also FK due to inheritance specialization)
4. PATIENT_ALLERGY.UserID → PATIENT.UserID
5. EMPLOYEE.UserID → USER.UserID (PK is also FK due to inheritance specialization)
6. MED_SIDE_EFFECT.MedicineID → MEDICINE.MedicineID
7. MED_HEALING_EFFECT.MedicineID → MEDICINE.MedicineID
8. ORDER.PatientID → PATIENT.UserID
9. ORDER.CashierID → EMPLOYEE.UserID
10. ORDER_ITEM.OrderID → ORDER.OrderID
11. ORDER_ITEM.MedicineID → MEDICINE.MedicineID
12. PRESCRIPTION.PatientID → PATIENT.UserID
13. PRESCRIPTION.PharmacistID → EMPLOYEE.UserID
14. PRESCRIPTION_ITEM.PrescriptionID → PRESCRIPTION.PrescriptionID
15. PRESCRIPTION_ITEM.MedicineID → MEDICINE.MedicineID
16. PAYMENT.OrderID → ORDER.OrderID

Appendix B: Sample SQL Scripts Overview

The project includes two primary SQL files: Creation.sql and Insertion.sql. Creation.sql defines the database, tables, constraints, and relationships. Insertion.sql seeds every table with a minimum of three rows to satisfy project requirements. Key SQL patterns observed include:

- IDENTITY(1,1) primary keys for surrogate identifiers (UserID, MedicineID, OrderID, etc.).
- CHECK constraints on enumerated status fields (e.g., CHK_Order_Status, CHK_Prescription_Status, CHK_Employee_JobType).
- DEFAULT constraints for financial and boolean fields (PatientBalance DEFAULT 0.0, IsActive DEFAULT 1, IsRefundable DEFAULT 0).
- Composite primary keys on associative and multi-valued tables (USER_ROLE, PATIENT_ALLERGY, MED_SIDE_EFFECT, ORDER_ITEM, PRESCRIPTION_ITEM).
- IF NOT EXISTS guards in Insertion.sql to prevent duplicate seed data on re-runs.

The full text of both scripts is included in the project source under /mnt/agents/upload/Creation.sql and /mnt/agents/upload/Insertion.sql.

--- End of Document ---